DLDCA Lab 5: Stack Overflow

Week 6

September 4, 2026

Introduction

This lab moves from integers to floating point, and introduces real functions. The tasks build up from scalar floating-point arithmetic, through writing and composing your own functions under the `SystemV` calling convention, to recursion, along with an additional graded task.

Necessary tools

We shall be using `nasm` to assemble x86-64 assembly into object files, and `ld/gcc/g++` to link those object files into an executable. Before starting the lab, ensure both are installed:

```
$ nasm -v
NASM version [version]
$ ld -v # or gcc/g++ --version
GNU ld (GNU Binutils for Ubuntu) [version]
```

Structure of submission/templates

Submission format:

```
your-roll-no/
|- labexam/
    |- poly_eval.s (TODO)
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of `your-roll-no.tar.gz` in lowercase. The command given below can be used to do the same

```
$ ./build_archive.sh
Enter Student Roll Number: your-roll-no
Packaging assignment files...
  [+] Copied: ...
Successfully created your-roll-no.tar.gz
```

Background: Scalar Floating Point & Functions

x86-64 provides sixteen 128-bit `xmm` registers. For this lab, we only care about the low 64 bits of each register (enough to hold a `double`) and only the *scalar double* (`sd`) instruction forms, such as `movsd`, `addsd`, and `mulsd`. The *packed* forms (`ps`/`pd`) operate on several values at once and will be covered in the next lab.

As a reminder, the following facts hold true about the `SystemV` convention:

1. Integer/pointer arguments and floating-point arguments are assigned from two *separate* sequences of registers – the first six integer/pointer arguments go in `rdi, rsi, rdx, rcx, r8, r9`, while the first eight floating-point arguments go in `xmm0` through `xmm7`, independently of each other. A function taking a `float` and an `int` would put the `float` in `xmm0` and the `int` in `rdi`.
2. Return values follow the same split: an integer/pointer result comes back in `rax`, a floating-point result in `xmm0`.
3. Every `xmm` register is caller-saved. There is no floating-point equivalent of a callee-saved general-purpose register like `rbx/r1[2-5]`. If you need a value in an `xmm` register to survive a `call`, you are responsible for saving it yourself (the stack is the usual place).
4. Syscalls are like special functions. They go to the `OS` and tell it to perform some action. To utilize them, we put the 'syscall number' in `rax` and put arguments in `rdi, rsi, rdx, r10, r8, r9`

Tasks

Lab Exam: Polynomial Evaluation

You are given an array of doubles, `coeffs`, its length `coeffs_len`, and a double `x_val`. Treating `coeffs` as the coefficients of a polynomial (`coeffs[0]` is the constant term, `coeffs[1]` the coefficient of x^1, and so on), evaluate

$$\texttt{coeffs}[0] \cdot x^0 + \texttt{coeffs}[1] \cdot x^1 + \cdots + \texttt{coeffs}[n-1] \cdot x^{n-1}$$

at $x =$ `x_val`, truncate the result toward zero, and exit with that as the exit code.

You'll implement this recursively, using Horner's method – which just means factoring x out repeatedly:

$$c_0 + x(c_1 + x(c_2 + \dots))$$

so that each step only needs one multiply and one add. As a recursive function over "the coefficients from some starting point onward", this is:

```
poly_eval(coeffs, n, x):
    if n == 1:
        return coeffs[0]

    rest = poly_eval(coeffs + 8, n - 1, x)
    return coeffs[0] + x * rest
```

Implement this as `poly_eval(double* coeffs{rdi}, size_t N{rsi}, x{xmm0})`.

Fill in `poly_eval` and `_start` in `poly_eval.s`.

To run your code:

```
$ make labexam
...
Return Code: 49
```

Table 1: Some basic x86-64 Assembly Instructions

Instruction	Operands	Effect
`mov`	`reg, reg/*mem/imm`	`reg` $\leftarrow$ `reg/*mem/imm`
`mov`	`*mem, reg`	`*mem` $\leftarrow$ `reg`
`movzx / movsx`	`reg, reg/*mem`	`reg` $\leftarrow$ `reg/*mem`, zero-/sign-extended
`lea`	`reg, *mem`	`reg` $\leftarrow$ address of `*mem`
`add`	`reg, reg/*mem/imm`	`reg` $\leftarrow$ `reg` $+$ `reg/*mem/imm`
`add`	`*mem, reg/imm`	`*mem` $\leftarrow$ `*mem` $+$ `reg/imm`
`sub`	`reg, reg/*mem/imm`	`reg` $\leftarrow$ `reg` $-$ `reg/*mem/imm`
`sub`	`*mem, reg/imm`	`*mem` $\leftarrow$ `*mem` $-$ `reg/imm`
`inc`	`reg/*mem`	`reg/*mem` $\leftarrow$ `reg/*mem` $+ 1$
`dec`	`reg/*mem`	`reg/*mem` $\leftarrow$ `reg/*mem` $- 1$
`neg`	`reg/*mem`	`reg/*mem` $\leftarrow -$`reg/*mem` (Two's complement)
`not`	`reg/*mem`	`reg/*mem` $\leftarrow \neg$`reg/*mem` (One's complement/bitwise)
`shl / shr`	`reg/*mem, imm/cl`	Logical shift left / right by `imm` or `cl` bits
`sar`	`reg/*mem, imm/cl`	Arithmetic shift right (preserves sign bit)
`imul`	`reg, reg/*mem/imm`	`reg` $\leftarrow$ `reg` $\times$ `reg/*mem/imm`
`imul`	`*mem, reg/imm`	`*mem` $\leftarrow$ `*mem` $\times$ `reg/imm`
`div`	`reg`	`rax` $\leftarrow$ `(rdx:rax)/reg` `rdx` $\leftarrow$ `(rdx:rax)`%`reg`
`and`	`reg, reg/*mem/imm`	`reg` $\leftarrow$ `reg` $\wedge$ `reg/*mem/imm`
`and`	`*mem, reg/imm`	`*mem` $\leftarrow$ `*mem` $\wedge$ `reg/imm`
`or`	`reg, reg/*mem/imm`	`reg` $\leftarrow$ `reg` $\vee$ `reg/*mem/imm`
`or`	`*mem, reg/imm`	`*mem` $\leftarrow$ `*mem` $\vee$ `reg/imm`
`xor`	`reg, reg/*mem/imm`	`reg` $\leftarrow$ `reg` $\oplus$ `reg/*mem/imm`
`xor`	`*mem, reg/imm`	`*mem` $\leftarrow$ `*mem` $\oplus$ `reg/imm`
`cmp`	`reg/*mem, reg/imm`	Sets flags based on subtraction: `reg/*mem - reg/imm`. ZF $=1$ if equal, SF $=1$ if result negative, CF $=1$ if borrow (unsigned), OF $=1$ if signed overflow.
`test`	`reg/*mem, reg/imm`	Sets flags based on bitwise AND: `reg/*mem` $\wedge$ `reg/imm`. ZF $=1$ if result is zero, SF $=1$ if high bit set, CF and OF are cleared. No result is stored.
`jmp`	`label`	Unconditional jump to label
`je / jz`	`label`	Jump if Zero Flag (ZF) $=1$
`jne / jnz`	`label`	Jump if Zero Flag (ZF) $=0$
`jl`	`label`	Jump if Less (SF $\neq$ OF)
`jle`	`label`	Jump if Less or Equal (ZF $=1$ or SF $\neq$ OF)
`jg`	`label`	Jump if Greater (ZF $=0$ and SF $=$ OF)
`jge`	`label`	Jump if Greater or Equal (SF $=$ OF)
`cmov`*cc*	`reg, reg/*mem`	`reg` $\leftarrow$ `reg/*mem` if *cc* holds, else unchanged.
`syscall`		Performs a syscall with syscall number stored in `rax` Arguments passed in other registers starting from `rdi`
`push`	`reg`	Pushes the value in `reg` onto the stack
`pop`	`reg`	Pops the value on top of the stack onto `reg`
`call`	`label`	Calls the function called `label`
`ret`		Pops the return address from stack and jumps to it

Table 2: Scalar Floating-Point (xmm) Instructions

Instruction	Operands	Effect
movsd	xmm, xmm/*mem	xmm $\leftarrow$ xmm/*mem (low 64 bits)
movsd	*mem, xmm	*mem $\leftarrow$ xmm (low 64 bits)
movq	xmm, reg/*mem	xmm $\leftarrow$ reg/*mem (raw 64 bits, no conversion)
movq	reg/*mem, xmm	reg/*mem $\leftarrow$ xmm (raw 64 bits, no conversion)
xorps / xorpd	xmm, xmm	xmm $\leftarrow$ xmm $\oplus$ xmm (idiom to zero an xmm register)
pxor	xmm, xmm	xmm $\leftarrow$ xmm $\oplus$ xmm (bitwise XOR on xmm)
addsd	xmm, xmm/*mem	xmm $\leftarrow$ xmm + xmm/*mem
subsd	xmm, xmm/*mem	xmm $\leftarrow$ xmm $-$ xmm/*mem
mulsd	xmm, xmm/*mem	xmm $\leftarrow$ xmm $\times$ xmm/*mem
divsd	xmm, xmm/*mem	xmm $\leftarrow$ xmm $\div$ xmm/*mem
comisd	xmm, xmm/*mem	Sets flags like cmp, comparing as doubles (use ja/jae/jb/jbe). Raises exception on NaN.
ucomisd	xmm, xmm/*mem	Same as comisd, but does not raise exception on NaN
cvttsd2si	reg, xmm/*mem	reg $\leftarrow$ integer part, truncated toward zero
cvtsi2sd[1]	xmm, reg/*mem	xmm $\leftarrow$ reg/*mem converted to a double

[1]Prefer loading floating point literals or the binary representation directly instead of using this. Use this only for runtime int to double conversions